Lecture 10: Sorting and Searching Algorithms

Sorting Algorithms

Two important classification criteria of sorting algorithms:

- Stable vs. unstable sorting
- In-place vs. out-of-place sorting

Sorting Stability:

- Stable sorting algorithms maintain the relative order of records with equal keys (i.e., values).
- Stable and unstable sorting procedures behave in the same way if there are no duplicates among the keys.
- A multitude of unique elements, for example, can be sorted with a stable or unstable sorting process; the result is always the same.
- Any sorting algorithm can be made stable by considering indexes as comparison parameter.

Stable sorting process by numbers:

```
1 Anton        1 Anton
4 Karl         1 Paul
3 Otto         3 Otto
5 Bernd   →    3 Herbert
3 Herbert      4 Karl
8 Alfred       5 Bernd
1 Paul         8 Alfred
```

Unstable sorting process by numbers:

```
1 Anton        1 Paul         1 Anton        1 Paul          1 Anton
4 Karl         1 Anton        1 Paul         1 Anton         1 Paul
3 Otto         3 Otto         3 Herbert      3 Herbert       3 Otto
5 Bernd   →    3 Herbert or   3 Otto    or   3 Otto      or   3 Herbert
3 Herbert      4 Karl         4 Karl         4 Karl          4 Karl
8 Alfred       5 Bernd        5 Bernd        5 Bernd         5 Bernd
1 Paul         8 Alfred       8 Alfred       8 Alfred        8 Alfred
```

If the sorting is unstable, Paul can stand in front of Anton or Herbert in front of Otto.

In-Place Sorting Algorithm:

An algorithm works **in-place** if, in addition to the memory required for storing the data to be processed, it only requires a constant amount of memory, i.e. independent of the amount of data to be processed. The algorithm overwrites the input data with the output data. Otherwise, the algorithm is called **out-of-place**, i.e. it requires additional storage space.

Bubble Sort:

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

Example: Sort the following set of elements **-** (5 1 4 2 8)

First Pass:
(**5 1** 4 2 8) –> (**1 5** 4 2 8), Here, algorithm compares the first two elements, and swaps since 5 > 1.
(1 **5 4** 2 8) –> (1 **4 5** 2 8), Swap since 5 > 4
(1 4 **5 2** 8) –> (1 4 **2 5** 8), Swap since 5 > 2
(1 4 2 **5 8**) –> (1 4 2 **5 8**), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass:
(**1 4** 2 5 8) –> (**1 4** 2 5 8)
(1 **4 2** 5 8) –> (1 **2 4** 5 8), Swap since 4 > 2
(1 2 **4 5** 8) –> (1 2 **4 5** 8)
(1 2 4 **5 8**) –> (1 2 4 **5 8**)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one **whole** pass without **any** swap to know it is sorted.

Third Pass:
(**1 2** 4 5 8) –> (**1 2** 4 5 8)
(1 **2 4** 5 8) –> (1 **2 4** 5 8)
(1 2 **4 5** 8) –> (1 2 **4 5** 8)
(1 2 4 **5 8**) –> (1 2 4 **5 8**)

Insertion Sort:

- A graphical example of insertion sort.
- The partial sorted list (black) initially contains only the first element in the list.
- With each iteration one element (red) is removed from the "not yet checked for order" input data and inserted in-place into the sorted list.

6 5 3 1 8 7 2 4

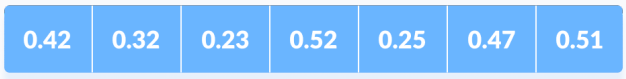
Bucket Sort:

- Bucket Sort is a sorting technique that sorts the elements by first dividing the elements into several groups called **buckets**.
- Several buckets are created. Each bucket is filled with a specific range of elements.
- The elements inside each **bucket** are sorted using any of the suitable sorting algorithms or recursively calling the same algorithm.
- Finally, the elements of the bucket are gathered to get the sorted array.

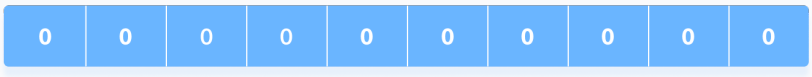
The process of bucket sort can be understood as **a scatter-gather** approach. The elements are first scattered into buckets then the elements of buckets are sorted. Finally, the elements are gathered in order.

How Bucket Sort Works?

1. Suppose the input array is:

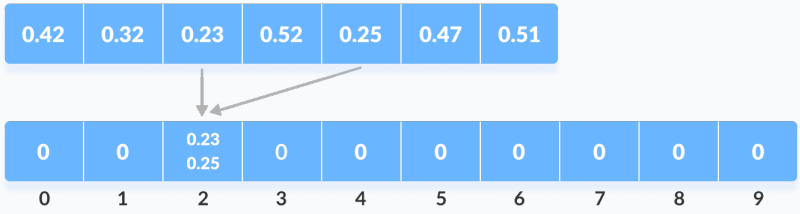


Create an array of size 10. Each slot of this array is used as a bucket for storing elements. (Array in which each position is a bucket)



2. Insert elements into the buckets from the array. The elements are inserted according to the range of the bucket.

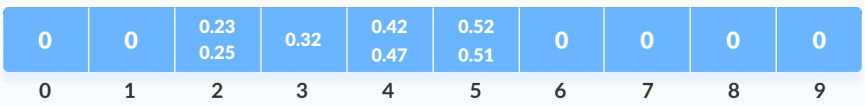
- In our example code, we have buckets each of ranges from 0 to 1, 1 to 2, 2 to 3,..... (n-1) to n.
- Suppose an input element is 0.23 is taken.
- It is multiplied by size = 10 (i.e. 0.23*10=2.3).
- Then, it is converted into an integer (i.e. 2.3 ≈ 2).
- Finally, 0.23 is inserted into **bucket-2**.



Similarly, 0.25 is also inserted into the same bucket. Every time, the floor value of the floating point number is taken.

If we take integer numbers as input, we have to divide it by the interval (10 here) to get the floor value.

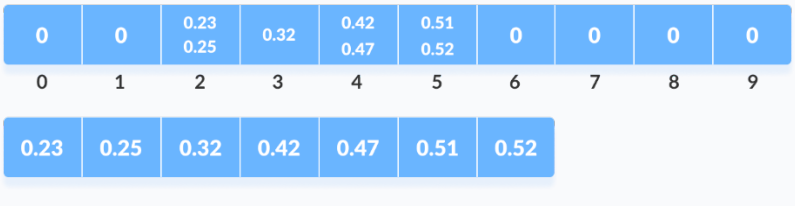
Similarly, other elements are inserted into their respective buckets.



3. The elements of each bucket are sorted using any of the stable sorting algorithms.

4. The elements from each bucket are gathered.

It is done by iterating through the bucket and inserting an individual element into the original array in each cycle. The element from the bucket is erased once it is copied into the original array.



Bucket sort is used when:

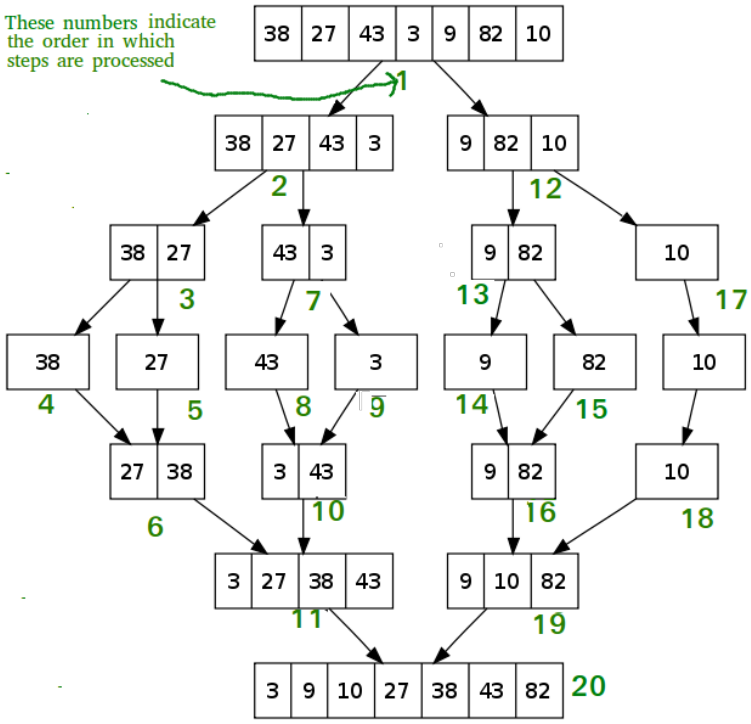
- input is uniformly distributed over a range.
- there are floating point values

Merge Sort:

- Merge Sort is a **Divide and Conquer** algorithm.
- It divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves.

Example:

- The following diagram shows the complete merge sort process for an example array {38, 27, 43, 3, 9, 82, 10}.
- If we take a closer look at the diagram, we can see that the array is recursively divided in two halves till the size becomes 1.
- Once the size becomes 1, the merge processes come into action and start merging arrays back till the complete array is merged.



Quick Sort:

- Quick Sort is also based on the concept of **Divide and Conquer**, just like merge sort.
- But in quick sort all the heavy lifting(major work) is done while **dividing** the array into subarrays, while in case of merge sort, all the real work happens during **merging** the subarrays.
- In case of quick sort, the combine step does absolutely nothing.

It is also called **partition-exchange sort**. This algorithm divides the list into three main parts:

1. Elements less than the **Pivot** element
2. Pivot element (Central element)
3. Elements greater than the pivot element

Pivot element can be any element from the array, it can be the first element, the last element or any random element.

How Quick Sort Works?

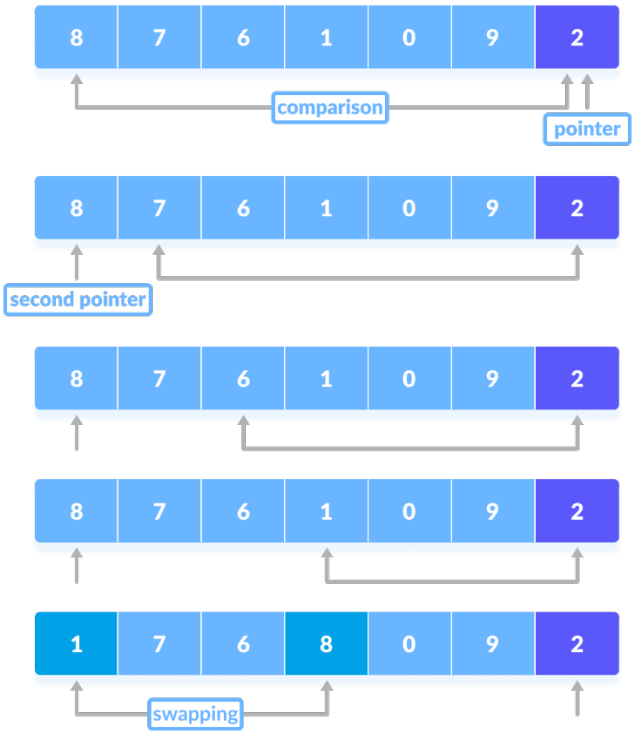
1. A pivot element is chosen from the array. You can choose any element from the array as the pivot element. Here, we have taken the rightmost (i.e., the last element) of the array as the pivot element.

2. The elements smaller than the pivot element are put on the left and the elements greater than the pivot element are put on the right

The above arrangement is achieved by the following steps.

   a. A pointer is fixed at the pivot element. The pivot element is compared with the elements beginning from the first index. If the element greater than the pivot element is reached, a second pointer is set for that element.

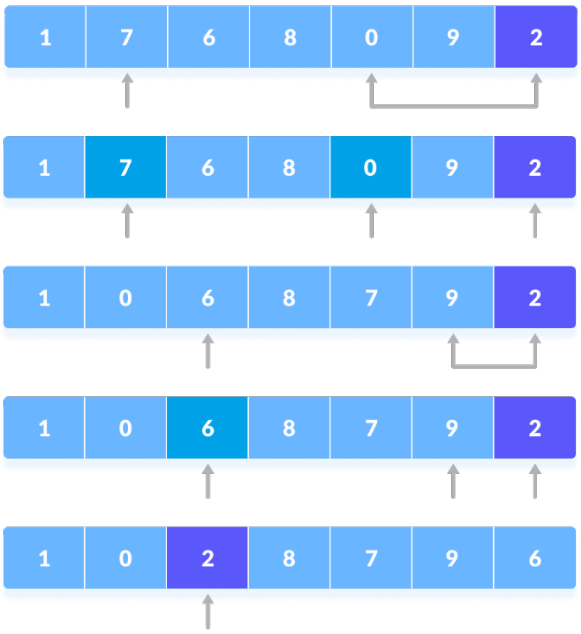
   b. Now, the pivot element is compared with the other elements (a third pointer). If an element smaller than the pivot element is reached, the smaller element is swapped with the greater element found earlier.



Comparison of pivot element with other elements

c. The process goes on until the second last element is reached.

Finally, the pivot element is swapped with the second pointer.



d. Now the left and right subparts of this pivot element are taken for further processing in the steps below.

3. Pivot elements are again chosen for the left and the right sub-parts separately. Within these sub-parts, the pivot elements are placed at their right position. Then, step 2 is repeated.

Select pivot element of in each half and put at correct place using recursion

4. The sub-parts are again divided into smaller sub-parts until each subpart is formed of a single element.

5. At this point, the array is already sorted.

Sorting Algorithm	**Best Case Complexity**	**Average Case Complexity**	**Worst Case Complexity**	**Worst Space Complexity**	**Advantage**	**Disadvantage**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	1) Simple 2) Stable 3) In-place 4) When the list is already sorted (best-case), the complexity of bubble sort is only O(n).	Not practical
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	1) Simple 2) Stable 3) In-place 4) When the list is already sorted (best-case), the complexity of bubble sort is only O(n).	Not practical
Bucket Sort	$O(n+k)$ (n: size of array; k: number of items)	$O(n+k)$ (n: size of array; k: number of items)	$O(n^2)$ (n: size of array)	$O(n)$	1) Fast 2) Stable	Out-of-place
Merge Sort	$O(n\ log(n))$	$O(n\ log(n))$	$O(n\ log(n))$	$O(n)$	1) Fast (also even in worst case)	Out-of-place

					2) Stable	
Quick Sort	$O(n\ log(n))$	$O(n\ log(n))$	$O(n^2)$	$O(log(n))$	1) Fast in practice 2) In-place	1) Not stable 2) Worst case with pivot element

Searching Algorithms:

Linear Search:

- Linear search is the simplest searching algorithm that searches for an element in a list in sequential order.
- We start at one end and check every element until the desired element is not found.

➢ Time Complexities
  - Best case complexity: $O(1)$
  - Average case complexity: $O(n/2)$
  - Worst case complexity: $O(n)$

➢ Worst case space complexity: $O(1)$

Binary Search:

- Binary Search is a searching algorithm for finding an element's position in a **sorted** array.
- In this approach, the element is always searched in the middle of a portion of an array.

- Time Complexities
  - Best case complexity: $O(1)$
  - Average case complexity: $O(log(n))$
  - Worst case complexity: $O(log(n))$
- Worst case space complexity: $O(n)$

References:

[1] Bucket Sort Algorithm (programiz.com)

[2] Merge Sort - GeeksforGeeks

[3] QuickSort Algorithm (programiz.com)